

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON (8 SEMANAS)

NIVEL 1 (100% PRINCIPIANTES ABSOLUTOS)

Curso 1: Fundamentos Básicos de Python

«De Cero a Programador: Los 4 Pilares Lógicos, Colecciones y Proyecto CLI»

Manual completo del curso compilado oficialmente. Incluye el desglose de las 8 clases semanales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor: William Rodríguez (Wisrovi)

Total Clases: 8 Semanas | Python: 3.10+ | Licencia: MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología: La Regla de la Bicicleta

Aprender a programar es pedalear en VS Code: la experimentación práctica diaria es la clave del dominio técnico.

Plan de Estudios de las 8 Semanas

Semana	Unidad Temática	Metáfora
CLASE 01	Clase 01: Panorama General y Filosofía de Python	«Python como Lenguaje de Comunicación Humano-Máquin...»
CLASE 02	Clase 02: Variables, Tipos de Datos y Operadores	«Variables como Cajas Etiquetadas en Memoria...»
CLASE 03	Clase 03: Control de Flujo: Condicionales (if / elif / else)	«Condicionales como Semáforos y Bifurcaciones en un...»
CLASE 04	Clase 04: Control de Flujo: Bucles (for / while)	«Bucles como una Cinta Transportadora de Fábrica...»
CLASE 05	Clase 05: Listas, Tuplas y Colecciones Básicas	«Listas como Archivadores Modulares y Tuplas como D...»
CLASE 06	Clase 06: Diccionarios y Conjuntos (Sets)	«Diccionarios como un Casillero con Llaves Únicas...»
CLASE 07	Clase 07: Funciones, Parámetros y Scope	«Funciones como Máquinas Reutilizables de una Fábri...»
CLASE 08	Clase 08: Proyecto Integrador: Sistema CLI Completo	«Construyendo tu Primera Aplicación Real de Consola...»

Clase 01: Panorama General y Filosofía de Python

Python es un lenguaje interpretado de alto nivel diseñado para maximizar la legibilidad y productividad del programador.

☀ Metáfora: Python como Lenguaje de Comunicación Humano-Máquina

Escribir en Python es como redactar instrucciones claras en un cuaderno que un asistente ejecuta al instante.

Principios Teóricos

El intérprete de Python lee el código de arriba a abajo, lo compila a bytecode y lo ejecuta en la máquina virtual (PVM).

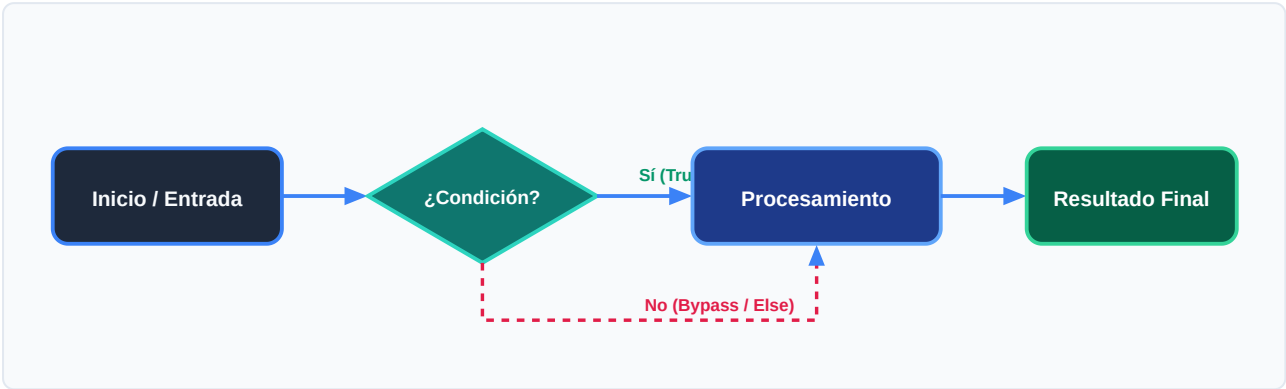
El 'Zen de Python' (PEP 20) establece que lo explícito es mejor que lo implícito y la legibilidad es prioritaria.

⚡ Regla de Oro

La indentación define la jerarquía lógica del código en Python.

Arquitectura y Movimiento de Datos

Flujo de lectura, compilación a bytecode en tiempo de ejecución y salida estándar.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Carga del archivo .py en memoria.	Código fuente en buffer de texto.
2. Proceso	Generación de Bytecode (.pyc).	Opcodes cargados en memoria.
3. Salida	Emisión a stdout (consola).	Buffer vaciado tras imprimir.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import sys

nombre = "Wisrovi Developer"
version = sys.version_info

print(f"Bienvenido {nombre} a Python {version.major}.{version.minor}")
print("Filosofía: Lo simple es mejor que lo complejo.")
```

⚠ Gotcha Frecuente

Mezclar espacios y tabulaciones genera un IndentationError silencioso.

❌ Antipatrón

```
def inicio(): print('Tab') print('Espacios') # ❌
IndentationError
```

✅ Patrón Correcto

```
def inicio(): print('Consistente') print('4 espacios')
# ✅ PEP 8
```

Clase 02: Variables, Tipos de Datos y Operadores

En Python, las variables no almacenan el dato directamente, sino una referencia a un objeto en el heap de memoria.

☀ Metáfora: Variables como Cajas Etiquetadas en Memoria

Una variable es una etiqueta adhesiva pegada a una caja; varias etiquetas pueden apuntar a la misma caja.

Principios Teóricos

Python es fuertemente tipado: no convierte tipos automáticamente sin orden explícita.

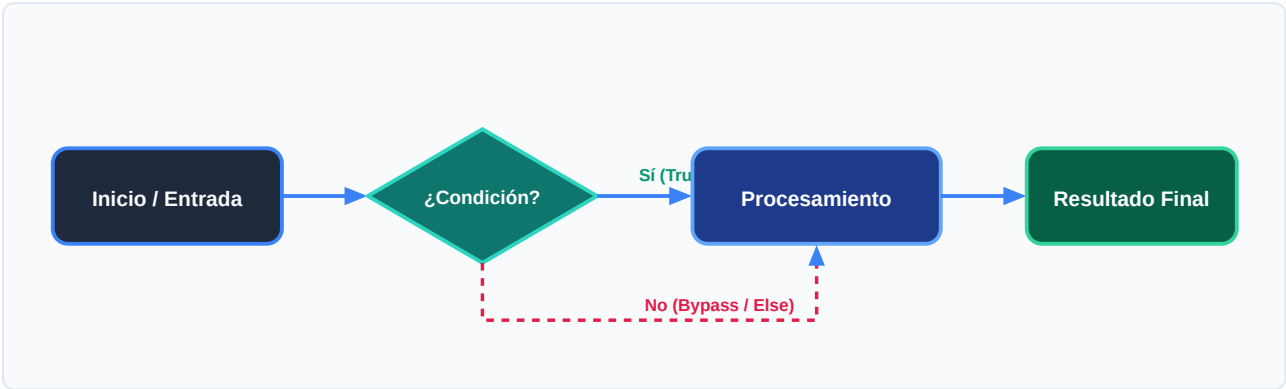
Los tipos primitivos (int, float, str, bool) son inmutables en memoria.

⚡ Regla de Oro

Convierte tipos explícitamente usando `int()` o `float()` antes de operar con entradas de usuario.

Arquitectura y Movimiento de Datos

Asignación de referencias en memoria y conversión de tipos primitivos.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Declaración y asignación de literales.	Creación del objeto en memoria.
2. Proceso	Enlace de la variable al identificador del objeto.	Puntero en el namespace local.
3. Salida	Evaluación de expresiones aritméticas.	Resultado en memoria temporal.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
edad: int = 28
precio: float = 19.99
nombre: str = "Wisrovi"
es_activo: bool = True

total = precio * 2
print(f"Usuario: {nombre} | Total a pagar: ${total:.2f}")
```

⚠ Gotcha Frecuente

input() siempre retorna un string; sumarlo directamente concatena texto.

❌ Antipatrón

```
edad = input('Edad: ') total = edad + 5 # ❌ TypeError
```

✅ Patrón Correcto

```
edad = int(input('Edad: ')) total = edad + 5 # ✅  
Correcto
```

Clase 03: Control de Flujo: Condicionales (if / elif / else)

Las estructuras condicionales permiten que tu programa tome decisiones autónomas basadas en condiciones booleanas.

☀ Metáfora: Condicionales como Semáforos y Bifurcaciones en un Tren

Un condicional es como una aguja ferroviaria que desvía el tren según el color del semáforo.

Principios Teóricos

Python evalúa las condiciones de forma secuencial; la primera rama que resulte True ejecuta su bloque.

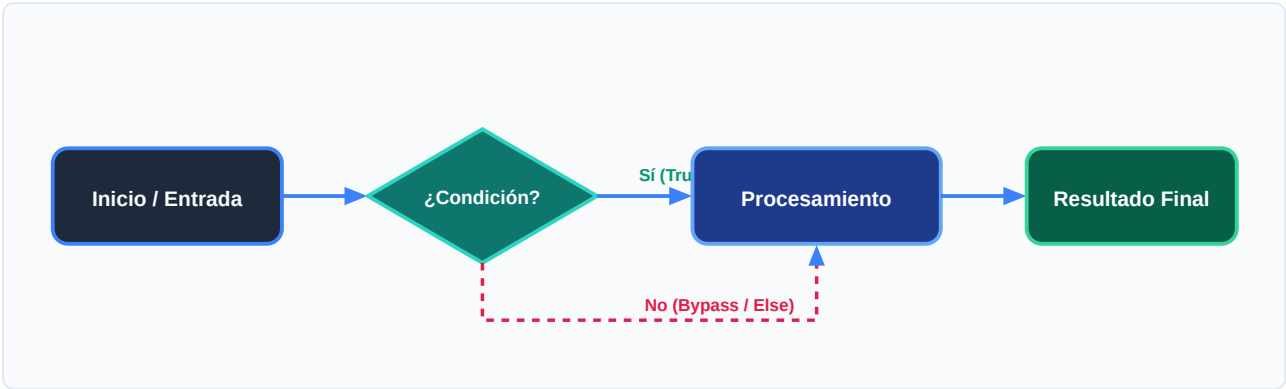
Cortocircuito booleano: En 'A and B', si A es False, B ni siquiera se evalúa.

⚡ Regla de Oro

Mantén las condiciones planas: evita anidar más de 3 niveles de if.

Arquitectura y Movimiento de Datos

Evaluación condicional de ramas múltiples (if - elif - else).



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Evaluación de la condición primaria (if).	Valor de verdad True/False.
2. Proceso	Desvío a rama elif en caso de False.	Paso a la siguiente condición.
3. Salida	Salida de la estructura hacia el flujo principal.	Continuación lineal.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
puntaje = 85

if puntaje >= 90:
    calificacion = "A - Excelente"
elif puntaje >= 80:
    calificacion = "B - Notable"
elif puntaje >= 70:
    calificacion = "C - Aprobado"
else:
    calificacion = "D - Refuerzo"

print(f"Resultado final: {calificacion}")
```

⚠ Gotcha Frecuente

Usar 'is' para comparar números o strings; 'is' compara direcciones de memoria.

❌ Antipatrón

if nombre is 'Juan': # ❌ SyntaxWarning

✅ Patrón Correcto

if nombre == 'Juan': # ✅ Comparación correcta

Clase 04: Control de Flujo: Bucles (for / while)

Los bucles permiten ejecutar un bloque de código múltiples veces sobre secuencias o hasta cumplir una condición.

☀ Metáfora: Bucles como una Cinta Transportadora de Fábrica

El bucle 'for' es como una cinta transportadora donde inspeccionas cada paquete uno a uno hasta terminar.

Principios Teóricos

El bucle 'for' en Python itera directamente sobre los elementos de cualquier objeto iterable.

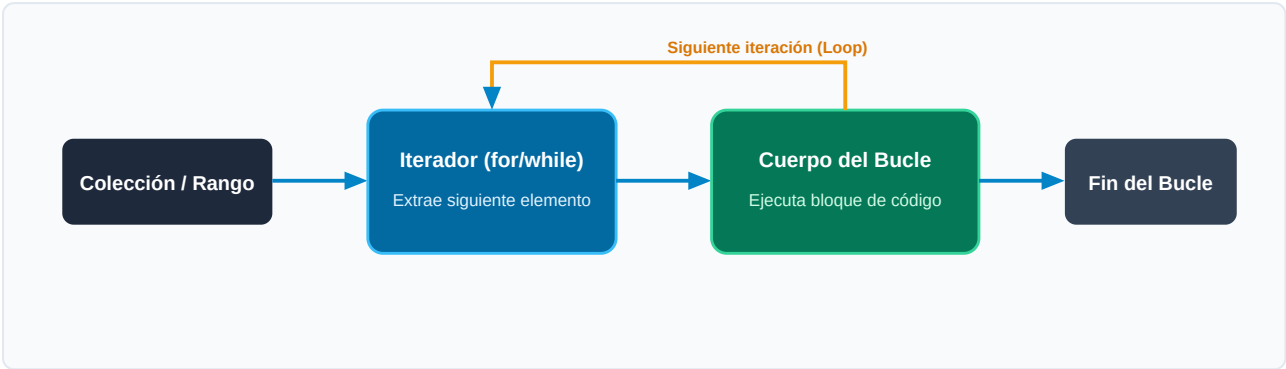
El bucle 'while' evalúa una condición antes de cada ciclo y se detiene cuando la condición es False.

⚡ Regla de Oro

En bucles while, asegúrate siempre de modificar la variable de control para evitar bucles infinitos.

Arquitectura y Movimiento de Datos

Ciclo de vida de una iteración con range e interrupción controlada.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Instanciación del objeto iterable (range).	Puntero en el primer índice.
2. Proceso	Extracción del elemento actual (next()).	Variable de iteración asignada.
3. Salida	Fin de colección o StopIteration.	Liberación del iterador.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
ventas = [120.0, 45.5, 300.0, 89.9]
total = 0.0

for venta in ventas:
    if venta < 50.0:
        continue
    total += venta

print(f"Total de ventas > $50: ${total:.2f}")
```

⚠ Gotcha Frecuente

Hacer `.remove()` en una lista dentro de un bucle `for` provoca saltos de elementos.

❌ Antipatrón

```
for n in numeros: if n % 2 == 0: numeros.remove(n) # ❌
```

Muta la colección

✅ Patrón Correcto

```
impares = [n for n in numeros if n % 2 != 0] # ✅ List comprehension
```

Clase 05: Listas, Tuplas y Colecciones Básicas

Las listas y tuplas son secuencias ordenadas que permiten almacenar conjuntos estructurados de datos.

☀ Metáfora: Listas como Archivadores Modulares y Tuplas como Documentos Notariados

Una lista es un archivador modular donde agregas carpetas; una tupla es un documento sellado inmutable.

Principios Teóricos

Las listas son mutables (su contenido cambia en memoria sin alterar su id).

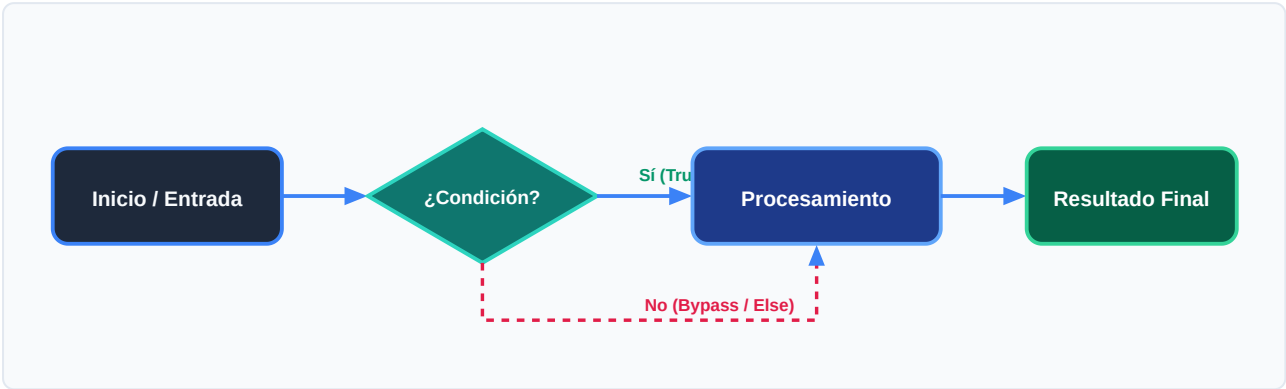
Las tuplas son inmutables y consumen menos memoria.

⚡ Regla de Oro

Si los datos representan una entidad fija que no debe cambiar, usa una tupla.

Arquitectura y Movimiento de Datos

Indexación, acceso por rebanadas (slicing) y mutabilidad.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Creación del arreglo dinámico de punteros.	Lista instanciada en el heap.
2. Proceso	Acceso por índice $O(1)$.	Lectura instantánea de la dirección.
3. Salida	Extracción de subconjuntos mediante slicing.	Nueva lista creada.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
inventario = ["Laptop", "Teclado", "Mouse"]
inventario.append("Monitor")
inventario.sort()

primeros_dos = inventario[:2]
print("Inventario ordenado:", inventario)
print("Top 2 productos:", primeros_dos)
```

⚠ Gotcha Frecuente

Hacer `lista_b = lista_a` no crea una copia, crea otro puntero a la misma lista.

❌ Antipatrón

`a = [1, 2, 3] b = a b.append(4) # ❌ Modifica también 'a'`

✅ Patrón Correcto

`a = [1, 2, 3] b = a.copy() # ✅ 'a' permanece intacta`

Clase 06: Diccionarios y Conjuntos (Sets)

Los diccionarios son colecciones asociativas basadas en pares clave-valor que permiten accesos ultra rápidos.

☀ Metáfora: Diccionarios como un Casillero con Llaves Únicas

Un diccionario es como un casillero: con tu llave (clave) abres instantáneamente el compartimento (valor).

Principios Teóricos

Las claves deben ser objetos inmutables y hashables (strings, números, tuplas).

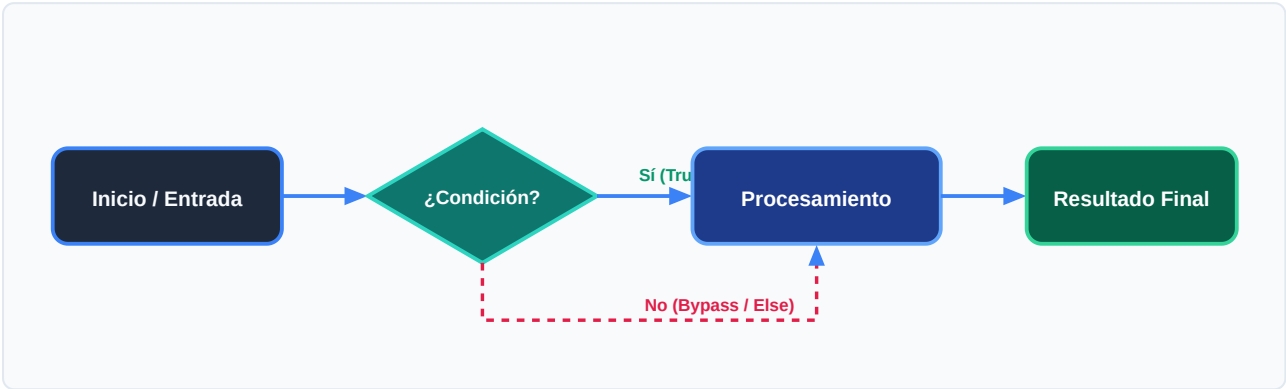
Los conjuntos (sets) son colecciones no ordenadas de elementos únicos.

⚡ Regla de Oro

Usa siempre `diccionario.get('clave', default)` para evitar excepciones `KeyError`.

Arquitectura y Movimiento de Datos

Hashing de claves, mapeo en tabla interna y operaciones de conjuntos.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Cálculo del hash mediante hash(key).	Hash entero generado.
2. Proceso	Indexación en la tabla hash interna.	Ubicación del bucket.
3. Salida	Iteración sobre items() o keys().	Vista dinámica generada.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
usuario = {
    "id": 101,
    "nombre": "Carlos Ruiz",
    "roles": {"admin", "editor"},
    "activo": True
}

email = usuario.get("email", "sin_correo@empresa.com")
print(f"Usuario: {usuario['nombre']} | Email: {email}")
```

⚠ Gotcha Frecuente

Hacer `data['no_existe']` lanza `KeyError` en lugar de devolver `None`.

❌ Antipatrón

```
data = {'a': 1} val = data['b'] # ❌ KeyError
```

✅ Patrón Correcto

```
data = {'a': 1} val = data.get('b', 0) # ✅ Seguro
```

Clase 07: Funciones, Parámetros y Scope

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica.

☀ Metáfora: Funciones como Máquinas Reutilizables de una Fábrica

Una función es como un electrodoméstico: introduces ingredientes (argumentos) y recibes el resultado (return).

Principios Teóricos

Principio DRY (Don't Repeat Yourself): Si repites código, conviértelo en una función.

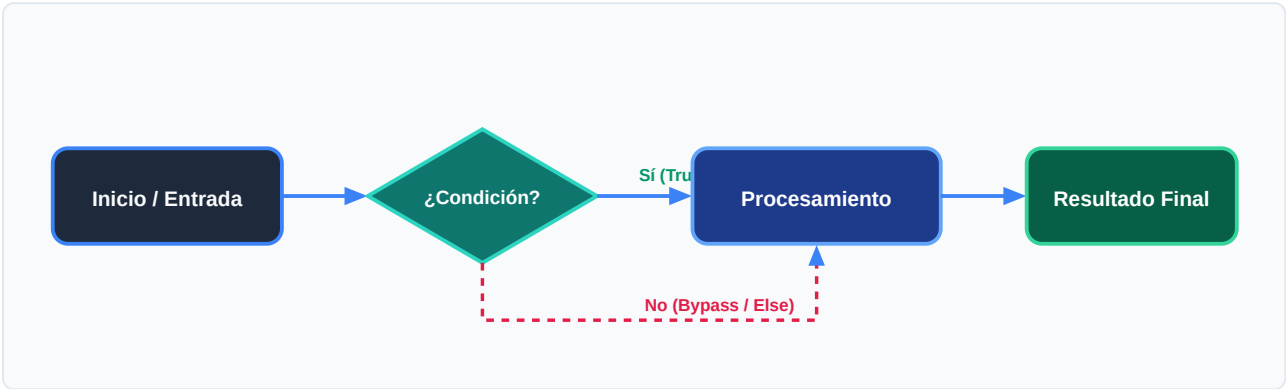
Regla de Scope LEGB: Python busca variables en orden: Local -> Enclosing -> Global -> Built-in.

⚡ Regla de Oro

Toda función debe tener una sola responsabilidad clara.

Arquitectura y Movimiento de Datos

Pila de llamadas (Call Stack), paso de argumentos y retorno de valores.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Definición y carga del objeto función.	Code object en memoria.
2. Proceso	Invocación y creación del Stack Frame.	Variables locales enlazadas.
3. Salida	Sentencia return y destrucción del frame.	Retorno entregado.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
def calcular_precio_final(base: float, descuento_pct: float = 0.0, iva_pct: float = 21.0) -> float:
    """Calcula el importe total tras aplicar descuento e impuestos."""
    subtotal = base * (1 - descuento_pct / 100)
    total = subtotal * (1 + iva_pct / 100)
    return round(total, 2)

print("Total:", calcular_precio_final(100.0, descuento_pct=10.0))
```

⚠ Gotcha Frecuente

Usar listas o diccionarios vacíos como valores por defecto en la firma.

✗ Antipatrón

```
def agregar(item, lista=[]): # ✗ Se comparte entre
    llamadas lista.append(item) return lista
```

✓ Patrón Correcto

```
def agregar(item, lista=None): # ✓ Inmutable None if
    lista is None: lista = [] lista.append(item) return
    lista
```


Clase 08: Proyecto Integrador: Sistema CLI Completo

El proyecto integrador une todos los conocimientos adquiridos en el Curso 1 para crear una herramienta real.

☀ Metáfora: Construyendo tu Primera Aplicación Real de Consola

Construir tu primera aplicación es como armar tu propia bicicleta: cada pieza encaja para ponerla en marcha.

Principios Teóricos

Arquitectura modular: Separación de la interfaz de consola de la lógica de negocio.

Manejo de excepciones: Asegurar que entradas inválidas no detengan el programa.

⚡ Regla de Oro

Estructura siempre tu punto de entrada con el patrón estándar `if __name__ == '__main__':`.

Arquitectura y Movimiento de Datos

Arquitectura del bucle principal de eventos del sistema CLI.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Inicialización del estado en memoria.	Colección de tareas activa.
2. Proceso	Despliegue del menú y captura de opción.	Espera de input en stdin.
3. Salida	Confirmación visual y repetición del ciclo.	Bucle activo hasta salir.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class TaskManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, title: str):
        self.tasks.append({"id": len(self.tasks) + 1, "title": title, "done": False})

    def list_tasks(self):
        return self.tasks

tm = TaskManager()
tm.add_task("Aprender Python con Wisrovi")
print("Tareas registradas:", tm.list_tasks())
```

⚠ Gotcha Frecuente

Escribir todo el código en un solo archivo plano sin funciones ni modularidad.

✗ Antipatrón

500 líneas de código plano desordenado ✗

✓ Patrón Correcto

Funciones modulares y clases con responsabilidades únicas ✓

Conclusiones Generales del Curso

Has completado las 8 semanas de **Curso 1: Fundamentos Básicos de Python**.



Certificación del Nivel

Dominio conceptual y práctico consolidado.

Bibliografía Canónica

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica	docs.python.org/3/
PEP 8	Guía de estilo	peps.python.org/pep-0008/
wisrovi SUITE	Librerías open source	github.com/wisrovi